

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

IMPACT ANALYSIS

Ingegneria del Software: Tecniche Avanzate – Impact Analysis

Mario Celzo	Matricola: NF22500193
Renato Mancino	Matricola: NF22500242
Alfonso Maria Ferrara	Matricola: NF22500210

Repository: `https://github.com/mariocelzo/biblioflow-app`

Anno Accademico 2025-2026

Indice

1 Introduzione **1**

1.1 Obiettivi del documento 1

1.2 Change Request Analizzate 1

1.3 Terminologia e Acronimi 2

2 Change Request 01 **3**

2.1 Descrizione della Change Request 3

2.2 Analisi Statica e Architetturale 3

2.2.1 Call Graph Architetturale (Stato Attuale) 3

2.2.2 Matrice di Raggiungibilità 3

2.3 Impact Analysis 6

2.3.1 Start Impact Set (SIS) 6

2.3.2 Candidate Impact Set (CIS) 7

2.4 Implementazione 8

2.4.1 Actual Impact Set (AIS) 8

2.4.2 False Positive Impact Set (FPIS) 9

2.4.3 Discovered Impact Set (DIS) 9

2.5 Metrics 10

3 Change Request 02 **12**

3.1 Descrizione della Change Request 12

3.2 Analisi Statica e Architetturale (Liberazione dei Posti) 12

3.2.1 Call Graph Architetturale (Stato Attuale) 12

3.2.2 Matrice di Raggiungibilità (Cross-Layer) 12

3.3 Impact Analysis 14

3.3.1 Start Impact Set (SIS) 14

3.3.2 Candidate Impact Set (CIS) 15

3.4 Implementazione 16

3.4.1 Actual Impact Set (AIS) 16

3.4.2 False Positive Impact Set (FPIS) 18

3.4.3 Discovered Impact Set (DIS) 18

3.5 Metrics 19

Elenco delle figure

2.1 Architettura Attuale (As-Is) del flusso di prenotazione. 4

3.1 Liberazione dei posti e canale realtime As-Is. 13

Elenco delle tabelle

1.1 Tabella riassuntiva delle Change Request 2

2.1 Dettaglio Change Request CR01 3
2.2 Matrice di Raggiungibilità tra le componenti del sistema (CR01) 5
2.3 Start Impact Set (SIS) per la CR01 6
2.4 Componenti aggiunti dal Candidate Impact Set per la CR01 7
2.5 False Positive Impact Set (FPIS) per la CR01 9
2.6 Discovered Impact Set (DIS) per la CR01 10

3.1 Dettaglio Change Request CR02 12
3.2 Matrice di Raggiungibilità Cross-Layer (CR02) 14
3.3 Start Impact Set (SIS) Dettagliato per CR02 14
3.4 Componenti aggiunti dal Candidate Impact Set per la CR02 15
3.5 False Positive Impact Set (FPIS) per la CR02 18
3.6 Discovered Impact Set (DIS) per la CR02 19

CAPITOLO 1

Introduzione

BiblioFlow è una piattaforma web per la prenotazione dei posti studio e la gestione dei servizi di una biblioteca universitaria. L'architettura combina un'interfaccia Next.js installabile come PWA, un insieme di route handler che accedono a PostgreSQL tramite Prisma e un servizio di automazioni periodiche che gestisce promemoria e posti non occupati.

Questo documento di Impact Analysis si propone di esaminare in via preventiva le modifiche richieste (Change Requests) per l'evoluzione della piattaforma. Lo scopo dell'analisi è stimare l'estensione degli interventi necessari sui moduli esistenti, permettendo così di pianificare lo sviluppo in modo accurato e di mitigare i rischi legati a potenziali effetti collaterali o regressioni nel codice. L'indagine viene svolta attraverso un'ispezione statica dei sorgenti della versione di partenza, volta a tracciare le dipendenze logiche tra i componenti.

1.1 Obiettivi del documento

L'analisi persegue i seguenti obiettivi operativi:

- **Mappatura Architetturale:** ricostruire la struttura delle dipendenze di BiblioFlow attraverso l'analisi statica, identificando i moduli critici e le interconnessioni tra pagine, route handler e servizi.
- **Stima dell'Impatto:** determinare per ogni Change Request l'insieme dei file che necessitano di modifica diretta e quelli esposti a rischio di propagazione.
- **Giustificazione Formale:** fornire evidenze che motivino l'inclusione dei vari componenti negli insiemi di impatto previsti.
- **Verifica a Posteriori:** definire i criteri quantitativi (metriche di Precision e Recall) per valutare, a fine implementazione, la discrepanza tra l'impatto ipotizzato in questa sede e le modifiche effettivamente realizzate.

1.2 Change Request Analizzate

Le richieste di modifica analizzate sono:

ID	Titolo	Descrizione Sintetica	Priorità
CR01	Prenotazione sicura e atomica dei posti studio	Identità e titolarità derivate dalla sessione autenticata, regole sugli orari centralizzate e creazione atomica delle prenotazioni, con un vincolo del database che impedisce due prenotazioni attive sovrapposte sullo stesso posto.	Alta

CR02	Lista d'attesa con riassegnazione automatica dei posti	Coda per posto e fascia oraria quando il posto è occupato; promozione automatica del primo in coda quando il posto si libera per cancellazione, no-show o promozione non confermata, con notifiche e tracciabilità.	Media

Tabella 1.1: Tabella riassuntiva delle Change Request

1.3 Terminologia e Acronimi

Per classificare i componenti software in base al loro coinvolgimento nelle modifiche, si adottano le seguenti definizioni standard:

- **SIS (Start Impact Set):** identifica i componenti che saranno oggetto di modifica diretta per soddisfare i requisiti della Change Request.
- **CIS (Candidate Impact Set):** comprende l'insieme esteso dei componenti che, pur non essendo modificati direttamente, potrebbero subire effetti indesiderati a causa di relazioni di dipendenza (import, chiamate HTTP, dati condivisi) con gli elementi del SIS.
- **AIS (Actual Impact Set):** rappresenta l'insieme dei componenti effettivamente alterati al termine della fase di implementazione.
- **FPIS (False Positive Impact Set):** sottoinsieme del CIS che, contrariamente alle previsioni, non ha richiesto interventi.
- **DIS (Discovered Impact Set):** componenti che hanno richiesto modifiche impreviste durante lo sviluppo, sfuggiti all'analisi iniziale.

CAPITOLO 2

Change Request 01

2.1 Descrizione della Change Request

Titolo	Prenotazione sicura e atomica dei posti studio
Situazione Attuale	La creazione di una prenotazione si fida dell'identificativo utente inviato dal client; lettura, cancellazione ed estensione non verificano la titolarità. Il controllo di disponibilità e la scrittura sono operazioni separate e il database non impedisce sovrapposizioni. Le regole sugli orari sono duplicate tra creazione ed estensione e accettano intervalli invertiti; check-in ed estensione confrontano date e orari in rappresentazioni incompatibili.
Situazione Desiderata	Identità e titolarità derivate dalla sessione, con risposte 401, 403 e 404 coerenti anche sulle route dello staff. Un servizio di dominio unico per la validazione degli intervalli. Creazione atomica in transazione, garantita da un vincolo di esclusione del database; il conflitto produce un 409 comprensibile.

Tabella 2.1: Dettaglio Change Request CR01

2.2 Analisi Statica e Architetturale

L'analisi statica del sistema attuale evidenzia una catena di esecuzione corta e lineare. Le pagine dello studente chiamano direttamente i route handler con `fetch`; prima di ogni route il middleware verifica soltanto che esista un cookie di sessione, senza leggerne il contenuto. Ciascun route handler contiene per intero la propria logica: legge e scrive con Prisma, e solo alcune route (il check-in e parte di quelle dello staff, ma non `src/app/api/admin/richieste/route.ts`) interrogano `src/lib/auth.ts` per conoscere l'utente. Non esiste uno strato di servizio condiviso tra le route di prenotazione: ognuna replica i propri controlli.

2.2.1 Call Graph Architetturale (Stato Attuale)

Di seguito è riportata la struttura delle chiamate tra i moduli coinvolti nel flusso di prenotazione, ricostruita tramite analisi del codice sorgente della versione di partenza: gli archi rappresentano un import, una chiamata HTTP di una pagina verso una route o il passaggio obbligato dal middleware.

2.2.2 Matrice di Raggiungibilità

La matrice seguente rappresenta la raggiungibilità tra i moduli del grafo precedente. I colori rappresentano la distanza topologica, cioè il numero di archi necessari per raggiungere un

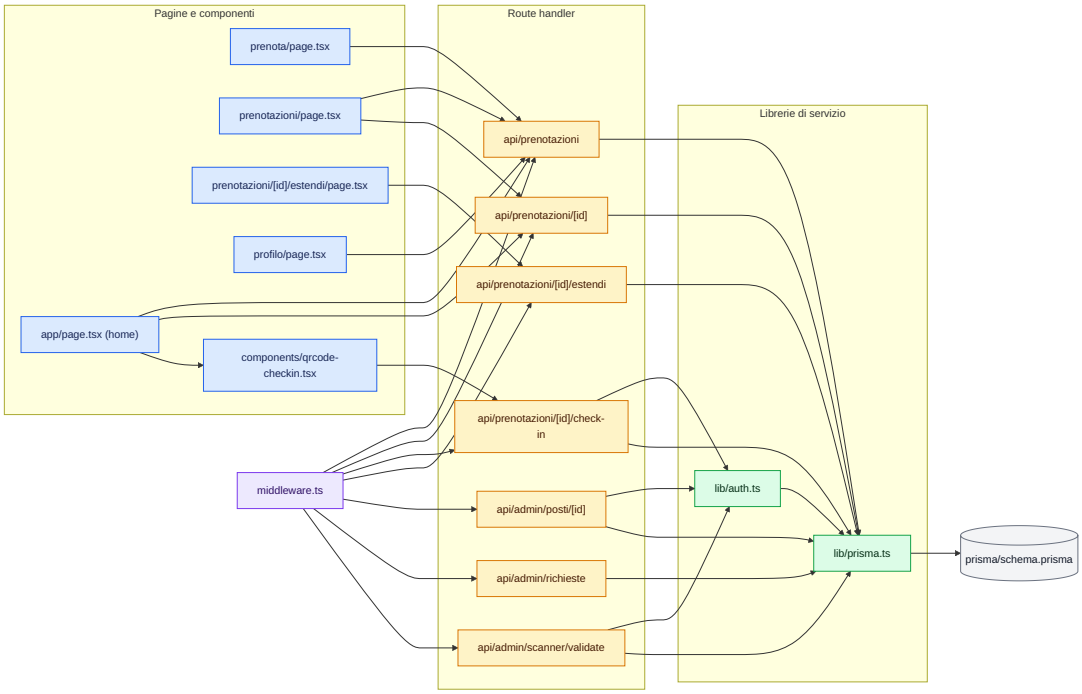


Figura 2.1: Architettura Attuale (As-Is) del flusso di prenotazione.

componente; il trattino indica che la destinazione non è raggiungibile.

Sorgente \ Destinazione	app/page	prenota/page	prenotazioni/page	estendi/page	profilo/page	qrcode-checkin	middleware	api/prenotazioni	api/pren./[id]	api/.../check-in	api/.../estendi	api/admin/posti/[id]	api/admin/richieste	api/admin/scanner	lib/auth
app/page	0	–	–	–	–	1	–	1	1	2	–	–	–	–	3
prenota/page	–	0	–	–	–	–	–	1	–	–	–	–	–	–	–
prenotazioni/page	–	–	0	–	–	–	–	1	1	–	–	–	–	–	–
estendi/page	–	–	–	0	–	–	–	–	–	–	1	–	–	–	–
profilo/page	–	–	–	–	0	–	–	1	–	–	–	–	–	–	–
qrcode-checkin	–	–	–	–	–	0	–	–	–	1	–	–	–	–	2
middleware	–	–	–	–	–	–	0	1	1	1	1	1	1	1	2
api/prenotazioni	–	–	–	–	–	–	–	0	–	–	–	–	–	–	–
api/pren./[id]	–	–	–	–	–	–	–	–	0	–	–	–	–	–	–
api/.../check-in	–	–	–	–	–	–	–	–	–	0	–	–	–	–	1
api/.../estendi	–	–	–	–	–	–	–	–	–	–	0	–	–	–	–
api/admin/posti/[id]	–	–	–	–	–	–	–	–	–	–	–	0	–	–	1
api/admin/richieste	–	–	–	–	–	–	–	–	–	–	–	–	0	–	–
api/admin/scanner	–	–	–	–	–	–	–	–	–	–	–	–	–	0	1
lib/auth	–	–	–	–	–	–	–	–	–	–	–	–	–	–	0
lib/prisma	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–
schema.prisma	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–

Sorgente \ Destinazione	lib/prisma	schema.prisma
app/page	2	3
prenota/page	2	3
prenotazioni/page	2	3
estendi/page	2	3
profilo/page	2	3
qrcode-checkin	2	3
middleware	2	3
api/prenotazioni	1	2
api/pren./[id]	1	2
api/.../check-in	1	2
api/.../estendi	1	2
api/admin/posti/[id]	1	2
api/admin/richieste	1	2
api/admin/scanner	1	2
lib/auth	1	2
lib/prisma	0	1
schema.prisma	–	0

Tabella 2.2: Matrice di Raggiungibilità tra le componenti del sistema (CR01)

La matrice mette in evidenza due punti di concentrazione: `src/lib/prisma.ts` e lo schema

sono raggiungibili da ogni route a distanza 1 e 2, quindi una modifica allo schema si propaga a tutto il sottosistema; il middleware raggiunge tutte le route a distanza 1, quindi un cambiamento nel modo di riconoscere l'utente potrebbe coinvolgerlo.

2.3 Impact Analysis

2.3.1 Start Impact Set (SIS)

Lo Start Impact Set (SIS) identifica le entità software che richiedono una modifica diretta per implementare i requisiti descritti nella Change Request. Per la CR01 sono stati individuati sette file che costituiscono il punto di iniezione della modifica.

File (Percorso)	Ruolo	Motivazione dell'Inclusione
`prisma/schema.prisma`	Modello dati	**Garanzia sul database:** la sovrapposizione è oggi controllata solo dal codice. Serve un vincolo che renda impossibili due prenotazioni attive sovrapposte sullo stesso posto.
`src/lib/auth.ts`	Sicurezza	**Identità dalla sessione:** mancano funzioni comuni per ottenere l'utente autenticato e verificarne ruolo e titolarità; ogni route le reimplementa o le omette.
`src/app/api/prenotazioni/route.ts`	Controller	**Creazione atomica:** legge `userId` dal corpo della richiesta ed esegue verifica e scrittura in due momenti distinti; l'elenco accetta un `userId` arbitrario come filtro.
`src/app/api/prenotazioni/[id]/route.ts`	Controller	**Titolarità:** lettura, cancellazione e check-in tramite `PATCH` non verificano che la prenotazione appartenga all'utente.
`src/app/api/prenotazioni/[id]/check-in/route.ts`	Controller	**Confronto data/ora:** confronta l'istante della richiesta con un orario `@db.Time` ancorato al 1970 e rifiuta check-in validi.
`src/app/api/prenotazioni/[id]/estendi/route.ts`	Controller	**Validazione duplicata:** non verifica la sessione, replica le regole sugli orari e calcola la durata tra date incompatibili.
`src/app/prenota/page.tsx`	Interfaccia	**Contratto della richiesta:** invia l'identificativo dell'utente nel corpo della prenotazione; il contratto con l'API deve cambiare.

Tabella 2.3: Start Impact Set (SIS) per la CR01

$$SIS = \left\{ \begin{array}{l} \texttt{prisma/schema.prisma}, \texttt{src/lib/auth.ts}, \\ \texttt{src/app/api/prenotazioni/route.ts}, \\ \texttt{src/app/api/prenotazioni/[id]/route.ts}, \\ \texttt{src/app/api/prenotazioni/[id]/check-in/route.ts}, \\ \texttt{src/app/api/prenotazioni/[id]/estendi/route.ts}, \\ \texttt{src/app/prenota/page.tsx} \end{array} \right\}$$

2.3.2 Candidate Impact Set (CIS)

L'analisi del Candidate Impact Set (CIS) mira a identificare i componenti che dipendono, direttamente o indirettamente, dagli elementi del SIS. Inizialmente l'indagine è stata condotta tramite ricerca testuale degli import e delle chiamate `fetch` nel codice sorgente, con il comando `grep`. Data la natura puramente sintattica di questo metodo, si è proceduto a una revisione manuale delle dipendenze, appoggiata al diagramma dei componenti e alla matrice ruoli-operazioni redatti sulla versione di partenza prima di intervenire sul codice.

A differenza di quanto avviene per un intervento isolato, il CIS non coincide con il SIS: la revisione ha aggiunto sei componenti.

File (Percorso)	Motivo della Propagazione
`src/middleware.ts`	Precede tutte le route e oggi riconosce l'utente dalla sola presenza del cookie: una gestione dell'identità basata sulla sessione potrebbe richiederne l'adeguamento.
`src/app/api/admin/posti/[id]/route.ts`	Restituisce le prenotazioni di un posto; la matrice ruoli-operazioni rileva che verifica la sessione ma non il ruolo dello staff.
`src/app/api/admin/richieste/route.ts`	Route dello staff che non verifica né la sessione né il ruolo: rientra nel requisito di risposte coerenti per l'area staff.
`src/app/api/admin/scanner/validate/route.ts`	Esegue il check-in tramite QR e scrive gli stessi stati della prenotazione toccati dalla CR.
`src/app/prenotazioni/page.tsx`	Consuma l'elenco e le operazioni sulla singola prenotazione, il cui contratto (filtri, codici di errore) può cambiare.
`src/app/prenotazioni/[id]/estendi/page.tsx`	Consuma la route di estensione, le cui regole e risposte saranno centralizzate.

Tabella 2.4: Componenti aggiunti dal Candidate Impact Set per la CR01

$$CIS = \left\{ \begin{array}{l} SIS \cup \{\texttt{src/middleware.ts}, \\ \quad \texttt{src/app/api/admin/posti/[id]/route.ts}, \\ \quad \texttt{src/app/api/admin/richieste/route.ts}, \\ \quad \texttt{src/app/api/admin/scanner/validate/route.ts}, \\ \quad \texttt{src/app/prenotazioni/page.tsx}, \\ \quad \texttt{src/app/prenotazioni/[id]/estendi/page.tsx}\} \end{array} \right\}$$

La revisione non ha incluso tre consumatori che pure chiamano route del SIS e compaiono nel call graph: la home (`src/app/page.tsx`), la pagina del profilo (`src/app/profilo/page.tsx`) e il componente `src/components/qrcode-checkin.tsx`. Nessuno dei tre è stato poi modificato: includerli avrebbe portato il CIS a 16 file senza cambiare l'intersezione con l'AIS.

2.4 Implementazione

2.4.1 Actual Impact Set (AIS)

L'Actual Impact Set è ricavato dal diff tra la versione di partenza (tag `baseline-pre-cr-bf-01`) e la chiusura delle Change Requests, limitato alle cartelle `src` e `prisma`: 36 file. Quattro sono estranei alle CR e sono esclusi: due correzioni richieste dal linter (`src/app/libri/[id]/page.tsx`, `src/components/accessibility/live-announcer.tsx`) e due migrazioni infrastrutturali (abilitazione della Row Level Security e prova della pipeline di deploy). Si considerano impattati solo i file con modifiche al codice: un file che ha ricevuto soltanto commenti non fa parte dell'AIS.

Durante la fase di implementazione, la scelta di raccogliere le regole di prenotazione in un servizio di dominio, invece di correggerle route per route, ha portato a una divergenza tra l'Impact Set stimato (CIS) e quello effettivo (AIS). L'AIS comprende i seguenti elementi:

- **File Modificati (dal CIS):**
  - `src/lib/auth.ts`: aggiunte `requireUser`, `requireRole` e `assertOwnership`, con un errore di autorizzazione tipizzato che le route traducono in 401, 403 o 404.
  - `src/app/api/prenotazioni/route.ts`, `src/app/api/prenotazioni/[id]/route.ts`, `src/app/api/prenotazioni/[id]/check-in/route.ts`, `src/app/api/prenotazioni/[id]/estendi/route.ts`: identità dalla sessione, titolarità e traduzione degli errori in risposte HTTP; creazione ed estensione delegano la validazione al servizio di dominio, il check-in combina data e orario prima del confronto.
  - `src/app/api/admin/posti/[id]/route.ts`, `src/app/api/admin/richieste/route.ts`: verifica del ruolo dello staff e validazione dei dati in ingresso.
- **Nuovi File Creati (Estensione):**
  - `src/lib/prenotazioni-service.ts`: servizio di dominio con la validazione degli intervalli e la creazione in transazione `Serializable`.
  - `src/lib/prenotazioni-errors.ts`: errori di dominio con codice stabile e messaggio comprensibile, tra cui il conflitto di disponibilità.
  - `prisma/migrations/..._vincolo_anti_sovrapposizione_prenotazioni`: vincolo di esclusione PostgreSQL sugli intervalli delle prenotazioni attive dello stesso posto.

Formalmente, l'insieme dei file impattati risulta essere:

$$AIS_{CR01} = \left\{ \begin{array}{l} \texttt{src/lib/auth.ts}, \texttt{src/app/api/prenotazioni/route.ts}, \\ \texttt{src/app/api/prenotazioni/[id]/route.ts}, \\ \texttt{src/app/api/prenotazioni/[id]/check-in/route.ts}, \\ \texttt{src/app/api/prenotazioni/[id]/estendi/route.ts}, \\ \texttt{src/app/api/admin/posti/[id]/route.ts}, \\ \texttt{src/app/api/admin/richieste/route.ts}, \\ \texttt{src/lib/prenotazioni-service.ts} \text{ (New)}, \\ \texttt{src/lib/prenotazioni-errors.ts} \text{ (New)}, \\ \text{migrazione del vincolo anti-sovrapposizione (New)} \end{array} \right\}$$

La differenza tra stima ed effettivo evidenzia un approccio additivo per la logica di dominio: le regole sono state spostate in moduli nuovi e le route si sono ridotte a tradurre input ed errori, invece di essere corrette una per una.

2.4.2 False Positive Impact Set (FPIS)

Il False Positive Impact Set (FPIS) è costituito dai componenti che erano stati stimati come impattati dalla modifica (membri del CIS), ma che non sono stati modificati per questa Change Request (non membri dell'AIS), anche se alcuni sono stati toccati in seguito dalla CR02. La tabella seguente descrive questi file e la giustificazione della loro esclusione.

File (Percorso)	Motivo dell'Esclusione
`prisma/schema.prisma`	Lo schema Prisma non può esprimere un vincolo di esclusione sugli intervalli: il vincolo è stato scritto in SQL in una migrazione dedicata e lo schema ha ricevuto solo un commento che vi rimanda.
`src/app/prenota/page.tsx`	La pagina continua a inviare l'identificativo dell'utente, ma il server lo ignora e usa quello della sessione: la correzione è stata fatta interamente lato server, senza cambiare il contratto della pagina.
`src/middleware.ts`	Le route ottengono l'utente dalla sessione per conto proprio; il middleware, che si limita a respingere le richieste prive di cookie, non ha richiesto modifiche per questa CR.
`src/app/api/admin/scanner/validate/route.ts`	Verificava già sessione, ruolo e firma del QR; la CR non ha introdotto nuovi stati della prenotazione che lo scanner dovesse riconoscere.
`src/app/prenotazioni/page.tsx`	I contratti di elenco e cancellazione sono rimasti compatibili: le nuove risposte di errore sono già gestite dal codice esistente.
`src/app/prenotazioni/[id]/estendi/page.tsx`	La route di estensione mantiene la stessa richiesta e la stessa risposta di successo; cambiano solo le regole lato server.

Tabella 2.5: False Positive Impact Set (FPIS) per la CR01

Analisi: i falsi positivi derivano da una stima prudente: ogni consumatore delle route di prenotazione è stato incluso perché il contratto poteva cambiare, e spostare la correzione lato server ha permesso di lasciare intatte le pagine. Lo schema è un caso simile a quello del file sostituito da una nuova classe: la modifica prevista è stata realizzata in un artefatto diverso, la migrazione SQL. Il middleware è stato poi rivisto durante la manutenzione successiva, quando è emerso che il solo controllo della presenza del cookie non era sufficiente: ora verifica davvero il token di sessione.

2.4.3 Discovered Impact Set (DIS)

Il Discovered Impact Set (DIS) comprende i file che non erano stati inclusi nell'insieme di impatto iniziale, ma che sono stati creati durante l'effettiva implementazione della Change Request. Questi componenti rappresentano il lavoro «scoperto» durante lo sviluppo, dovuto a decisioni architetturali prese per migliorare la manutenibilità del codice.

Le seguenti classi compongono il DIS per la CR01:

File (Percorso)	Motivazione dell'Inclusione
`src/lib/prenotazioni-service.ts`	Invece di correggere separatamente le regole sugli orari duplicate in creazione ed estensione, è stato creato un servizio di dominio indipendente da richiesta e risposta HTTP: validazione degli intervalli, verifica della sala e del posto, creazione in transazione `Serializable`; la route di estensione usa la stessa validazione in una propria transazione.
`src/lib/prenotazioni-errors.ts`	Le route devono distinguere i casi d'errore senza conoscere il database: gli errori di dominio portano un codice stabile e un messaggio comprensibile, che la route traduce nel codice HTTP corretto.
Migrazione del vincolo anti-sovrapposizione	Lo schema Prisma non consente di dichiarare un vincolo di esclusione sugli intervalli: è stato necessario scriverlo in SQL in una migrazione dedicata, al posto della modifica di `schema.prisma` prevista nel SIS.

Tabella 2.6: Discovered Impact Set (DIS) per la CR01

Analisi: la presenza di questi file nel DIS mostra che una parte dell'impatto si è spostata dalla modifica del codice esistente alla creazione di moduli nuovi. È lo stesso fenomeno che alimenta l'FPIS: concentrando le regole in un punto solo, i consumatori previsti nel CIS sono rimasti intatti.

2.5 Metrics

In questa sezione valutiamo l'accuratezza della stima dell'impatto utilizzando le metriche di Precision e Recall.

- **Candidate Impact Set (CIS):** 13 file
- **Actual Impact Set (AIS):** 10 file
- **Intersezione (CIS ∩ AIS):** 7 file

Precision (Precisione) Misura la frazione di file previsti che sono stati effettivamente modificati.

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{7}{13} \approx 0{,}54\ (53{,}8\%)$$

Un valore del 54% indica che poco più della metà dei file previsti è stata coinvolta. I sei falsi positivi sono tre pagine il cui contratto è rimasto compatibile, il middleware (rivisto solo nella manutenzione successiva), la route dello scanner, che già verificava sessione e ruolo, e lo schema, la cui modifica è stata realizzata nella migrazione SQL.

Recall (Richiamo) Misura la frazione di file effettivamente modificati che erano stati previsti correttamente all'inizio.

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{7}{10} = 0{,}70\ (70\%)$$

Un valore del 70% indica che oltre due terzi del lavoro effettivo erano stati previsti; la parte restante corrisponde ai tre file nuovi del DIS: servizio ed errori di dominio, nati dalla scelta di centralizzare le regole, e la migrazione SQL del vincolo di esclusione, che lo schema Prisma non può esprimere.

CAPITOLO 3

Change Request 02

3.1 Descrizione della Change Request

Titolo	Lista d'attesa con riassegnazione automatica dei posti
Situazione Attuale	Se il posto è occupato nella fascia scelta lo studente riceve solo un errore. Quando una prenotazione viene annullata o il titolare non si presenta, il posto torna disponibile senza avvisare gli altri studenti; l'automazione periodica segna il no-show, libera il posto e avvisa soltanto il titolare.
Situazione Desiderata	Ingresso in coda per posto e fascia dalla mappa, dalla griglia mobile o dal messaggio di conflitto; ordine di arrivo senza duplicati; promozione automatica del primo in coda su cancellazione (studente o staff), no-show e promozione non confermata, con notifica, scadenza della promozione e tracciabilità completa.

Tabella 3.1: Dettaglio Change Request CR02

3.2 Analisi Statica e Architetturale (Liberazione dei Posti)

L'architettura coinvolta dalla CR02 è più distribuita di quella della CR01: un posto può liberarsi per quattro vie indipendenti (cancellazione dello studente, cancellazione dello staff, annullamento manuale dei no-show dalla pagina delle anomalie, automazione periodica) che oggi non condividono alcun codice. L'analisi statica ha inoltre ricostruito la catena dei componenti di aggiornamento in tempo reale, presenti nel codice ma non collegati all'interfaccia: nessuna pagina usa l'hook `src/hooks/use-sse.ts` e nessun modulo invoca `src/lib/realtime-events.ts`.

3.2.1 Call Graph Architetturale (Stato Attuale)

Per comprendere l'impatto della lista d'attesa è stata mappata la catena che va dalle interfacce ai punti in cui un posto si libera, fino all'accesso ai dati. Il diagramma adotta una prospettiva orientata al flusso dei dati e mostra il disaccoppiamento tra i percorsi di liberazione e il canale in tempo reale.

3.2.2 Matrice di Raggiungibilità (Cross-Layer)

La seguente matrice quantifica la distanza logica tra i livelli dell'architettura; ogni cella riporta la distanza minima tra un modulo del livello di partenza e uno del livello di arrivo. Nel contesto della CR02 è significativo ciò che *non* è raggiungibile: dall'interfaccia non si arriva

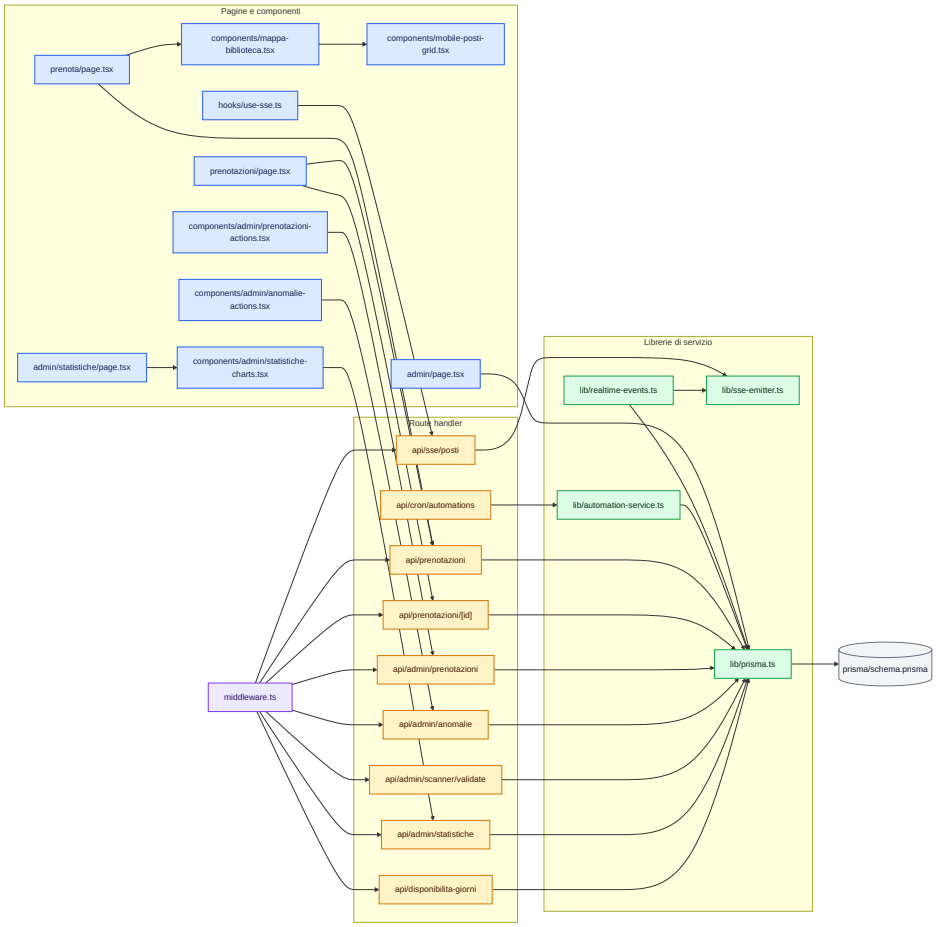


Figura 3.1: Liberazione dei posti e canale realtime As-Is.

al servizio delle automazioni né al canale SSE, quindi l'esito di una promozione non ha oggi alcuna strada per tornare all'utente se non tramite la base di dati e le notifiche.

Sorgente \ Destinazione	UI prenotazione	UI staff	Client realtime	Middleware	Route handler	Cron	automation-service	Canale SSE	Accesso ai dati
UI prenotazione	0	–	–	–	1	–	–	–	2
UI staff	–	0	–	–	1	–	–	–	1
Client realtime	–	–	0	–	–	–	–	1	–
Middleware	–	–	–	0	1	–	–	1	2
Route handler	–	–	–	–	0	–	–	–	1
Cron	–	–	–	–	–	0	1	–	2
automation-service	–	–	–	–	–	–	0	–	1
Canale SSE	–	–	–	–	–	–	–	0	1
Accesso ai dati	–	–	–	–	–	–	–	–	0

Tabella 3.2: Matrice di Raggiungibilità Cross-Layer (CR02)

3.3 Impact Analysis

3.3.1 Start Impact Set (SIS)

Lo Start Impact Set per la CR02 comprende i componenti che possiedono i principali punti in cui un posto si libera e il modello dei dati che deve rappresentare la coda.

File (Percorso)	Ruolo	Motivazione dell'Inclusione
`prisma/schema.prisma`	Modello dati	**Nuova entità:** la richiesta in coda deve essere persistita con utente, posto, intervallo, stato e ordine di arrivo, impedendo i duplicati attivi.
`src/lib/automation-service.ts`	Servizio	**No-show e scadenze:** il rilascio automatico dei posti non occupati è uno degli eventi di liberazione; qui va avviata la promozione e gestita la scadenza delle promozioni non confermate.
`src/app/api/admin/prenotazioni/route.ts`	Controller	**Cancellazione dello staff:** l'annullamento di una prenotazione da parte dello staff libera il posto e deve tentare la promozione.
`src/app/api/prenotazioni/[id]/route.ts`	Controller	**Cancellazione dello studente:** l'annullamento da parte del titolare è il caso più frequente di posto liberato.

Tabella 3.3: Start Impact Set (SIS) Dettagliato per CR02

$$SIS = \left\{ \begin{array}{l} \texttt{prisma/schema.prisma}, \texttt{src/lib/automation-service.ts}, \\ \texttt{src/app/api/admin/prenotazioni/route.ts}, \\ \texttt{src/app/api/prenotazioni/[id]/route.ts} \end{array} \right\}$$

3.3.2 Candidate Impact Set (CIS)

Il Candidate Impact Set identifica i componenti che potrebbero subire effetti collaterali a causa delle modifiche introdotte nel SIS. Come per la CR01, l'indagine è partita dalla ricerca testuale di import, chiamate e tipi condivisi (stati della prenotazione, tipi di notifica e di evento) ed è stata completata con una revisione manuale. A differenza di un intervento isolato, la propagazione è ampia: i nuovi stati ed eventi attraversano statistiche, anomalie, interfacce e automazioni.

File (Percorso)	Motivo della Propagazione
`src/app/prenota/page.tsx`, `src/components/mappa-biblioteca.tsx`, `src/components/mobile-posti-grid.tsx`	Sono i punti in cui lo studente incontra il posto occupato: la proposta di entrare in coda deve comparire qui.
`src/app/api/cron/automations/route.ts`	Invoca il servizio delle automazioni: esecuzioni concorrenti potrebbero promuovere due volte lo stesso utente.
`src/app/api/admin/anomalie/route.ts`, `src/app/admin/page.tsx`	Contano no-show e cancellazioni per dashboard e anomalie: i nuovi stati ed eventi potrebbero alterarne i valori.
`src/app/api/admin/statistiche/route.ts`, `src/app/admin/statistiche/page.tsx`	Le statistiche aggregano gli stati delle prenotazioni; la coda richiede un indicatore proprio senza inquinare quelli esistenti.
`src/app/api/admin/scanner/validate/route.ts`	Esegue il check-in: deve accettare le prenotazioni nate da una promozione e non le richieste ancora in coda.
`src/app/api/disponibilita-giorni/route.ts`	Calcola i giorni prenotabili dagli stati delle prenotazioni.
`src/lib/realtime-events.ts`, `src/lib/sse-emitter.ts`, `src/app/api/sse/posti/route.ts`, `src/hooks/use-sse.ts`	Canale previsto per avvisare in tempo reale dei posti liberati e delle promozioni.

Tabella 3.4: Componenti aggiunti dal Candidate Impact Set per la CR02

$$CIS = \left\{ \begin{array}{l} SIS \cup \{\texttt{src/app/prenota/page.tsx}, \\ \quad \texttt{src/components/mappa-biblioteca.tsx}, \\ \quad \texttt{src/components/mobile-posti-grid.tsx}, \\ \quad \texttt{src/app/api/cron/automations/route.ts}, \\ \quad \texttt{src/app/api/admin/anomalie/route.ts}, \texttt{src/app/admin/page.tsx}, \\ \quad \texttt{src/app/api/admin/statistiche/route.ts}, \\ \quad \texttt{src/app/admin/statistiche/page.tsx}, \\ \quad \texttt{src/app/api/admin/scanner/validate/route.ts}, \\ \quad \texttt{src/app/api/disponibilita-giorni/route.ts}, \\ \quad \texttt{src/lib/realtime-events.ts}, \texttt{src/lib/sse-emitter.ts}, \\ \quad \texttt{src/app/api/sse/posti/route.ts}, \texttt{src/hooks/use-sse.ts}\} \end{array} \right\}$$

Il CIS conta quindi 18 file: i 4 del SIS e i 14 elencati.

3.4 Implementazione

3.4.1 Actual Impact Set (AIS)

Con lo stesso criterio della CR01 (diff fino alla chiusura delle Change Requests, solo modifiche al codice), durante l'implementazione della CR02 le scelte architetturali hanno privilegiato il riuso del servizio di dominio creato per la CR01: ingresso in coda, annullamento, posizione e promozione sono funzioni dello stesso servizio. L'AIS comprende i seguenti elementi:

- **File Modificati (dal CIS):**
  - `prisma/schema.prisma`: entità `ListaAttesa` con stati `IN_ATTESA`, `PROMOSSA`, `SCADUTA`, `ANNULLATA` e nuovi tipi di notifica e di evento.
  - `src/lib/automation-service.ts`: promozione dopo il no-show, scadenza delle promozioni non confermate entro quindici minuti, protezione dalle doppie esecuzioni.
  - `src/app/api/cron/automations/route.ts`: lock di PostgreSQL che impedisce due esecuzioni contemporanee.
  - `src/app/api/admin/prenotazioni/route.ts`: promozione dopo l'annullamento dello staff ed esito restituito all'interfaccia.
  - `src/app/prenota/page.tsx`, `src/components/mappa-biblioteca.tsx`, `src/components/mobile-posti-grid.tsx`: proposta di ingresso in coda sul posto occupato e nel messaggio di conflitto.
  - `src/app/api/admin/statistiche/route.ts`, `src/app/admin/statistiche/page.tsx`: indicatore dedicato alla coda.
  - `src/lib/realtime-events.ts`: nuove funzioni `emitCodaIngresso` ed `emitCodaPromozione`; solo la seconda è invocata, dall'annullamento dello staff.
- **File Modificati fuori stima:**

  - `src/lib/prenotazioni-service.ts`: nuove funzioni di dominio per la coda (`entraInCoda`, `annullaRichiestaCoda`, `posizioneInCoda`, `promuoviPrimoInCoda`).
  - `src/lib/prenotazioni-errors.ts`: errori della coda e indicazione `suggerisciCoda` sul conflitto di disponibilità.
  - `src/app/notifiche/page.tsx`: resa dei nuovi tipi di notifica.
  - `src/middleware.ts`: dichiarazione esplicita della route della coda nel matcher.
  - `src/components/admin/prenotazioni-actions.tsx`, `src/components/admin/statistiche-charts.tsx`: esito della promozione per lo staff, grafico della coda.

- **Nuovi File Creati (Estensione):**
  - `src/app/api/prenotazioni/coda/route.ts`: elenco, ingresso e uscita dalla coda per lo studente.
  - `src/hooks/use-ingresso-coda.ts`: logica di ingresso in coda condivisa tra mappa e griglia.
  - `src/lib/eventi-coda.ts`: elenco dei tipi di evento della coda da escludere dalle metriche esistenti.
  - `src/app/notifiche/tipo-config.tsx`: configurazione di icone e colori per tutti i tipi di notifica, con ripiego per i tipi sconosciuti.
  - Due migrazioni: tabella della lista d'attesa con indice FIFO e indice unico sulle richieste attive; nuovi valori degli enum.

Formalmente, l'insieme dei file impattati risulta essere:

$$AIS_{CR02} = \left\{ \begin{array}{l} \texttt{prisma/schema.prisma}, \texttt{src/lib/automation-service.ts}, \\ \texttt{src/app/api/cron/automations/route.ts}, \\ \texttt{src/app/api/admin/prenotazioni/route.ts}, \\ \texttt{src/app/prenota/page.tsx}, \texttt{src/components/mappa-biblioteca.tsx}, \\ \texttt{src/components/mobile-posti-grid.tsx}, \\ \texttt{src/app/api/admin/statistiche/route.ts}, \\ \texttt{src/app/admin/statistiche/page.tsx}, \texttt{src/lib/realtime-events.ts}, \\ \texttt{src/lib/prenotazioni-service.ts}, \\ \texttt{src/lib/prenotazioni-errors.ts}, \\ \texttt{src/app/notifiche/page.tsx}, \texttt{src/middleware.ts}, \\ \texttt{src/components/admin/prenotazioni-actions.tsx}, \\ \texttt{src/components/admin/statistiche-charts.tsx}, \\ \texttt{src/app/api/prenotazioni/coda/route.ts} \text{ (New)}, \\ \texttt{src/hooks/use-ingresso-coda.ts} \text{ (New)}, \\ \texttt{src/lib/eventi-coda.ts} \text{ (New)}, \\ \texttt{src/app/notifiche/tipo-config.tsx} \text{ (New)}, \\ \text{migrazione della lista d'attesa (New)}, \\ \text{migrazione di estensione degli enum (New)} \end{array} \right\}$$

3.4.2 False Positive Impact Set (FPIS)

Il False Positive Impact Set evidenzia i componenti inclusi nel CIS che sono rimasti inalterati durante l'implementazione.

File (Percorso)	Motivo dell'Esclusione
`src/app/admin/page.tsx`	La coda è un'entità separata: le richieste in attesa non sono prenotazioni e i conteggi della dashboard non sono cambiati.
`src/app/api/admin/scanner/validate/route.ts`	Una promozione produce una normale prenotazione `CONFERMATA`: lo scanner la riconosce senza modifiche, e una richiesta in coda non ha un QR.
`src/lib/sse-emitter.ts`	I nuovi eventi sono stati aggiunti in `src/lib/realtime-events.ts`; il meccanismo di consegna è rimasto invariato.
`src/app/api/sse/posti/route.ts`, `src/hooks/use-sse.ts`	Hanno ricevuto solo commenti che documentano i nuovi eventi: il canale non è stato collegato all'interfaccia.
`src/app/api/disponibilita-giorni/route.ts`	I giorni prenotabili dipendono dal calendario e dalle prenotazioni attive, non dalla coda.
`src/app/api/prenotazioni/[id]/route.ts`	**Omissione, non vero falso positivo.** A fine CR la cancellazione da parte dello studente non avviava la promozione: il caso era verificato solo a livello di servizio. Il difetto è emerso dalla verifica dal vivo ed è stato corretto nella manutenzione successiva, collegando la route alla promozione.
`src/app/api/admin/anomalie/route.ts`	**Omissione, non vero falso positivo.** L'azione dello staff «Annulla prenotazioni senza check-in» segna il no-show e libera il posto senza avviare la promozione; è stata collegata alla coda nella manutenzione successiva, delegando a `releaseNoShowReservations`.

Tabella 3.5: False Positive Impact Set (FPIS) per la CR02

Analisi: sei degli otto elementi sono veri falsi positivi. Dashboard, scanner e giorni prenotabili non sono cambiati grazie alla scelta di modellare la coda come entità separata invece che come nuovo stato della prenotazione; emettitore, route SSE e hook non sono cambiati perché il canale in tempo reale non è stato collegato all'interfaccia. Gli altri due sono un caso diverso e più istruttivo: la stima era corretta, l'implementazione incompleta. Per questo l'FPIS non va letto come misura della sola qualità della stima.

3.4.3 Discovered Impact Set (DIS)

Il Discovered Impact Set comprende i file non inclusi nel CIS che sono stati creati o modificati durante l'implementazione.

File (Percorso)	Motivazione dell'Inclusione
`src/lib/prenotazioni-service.ts`	Le operazioni di coda devono condividere la transazione e le regole della prenotazione: sono state aggiunte al servizio di dominio della CR01, con lock sulle righe per impedire doppie promozioni concorrenti.
`src/lib/prenotazioni-errors.ts`	Il conflitto di disponibilità deve dire all'interfaccia che può proporre la coda, e le operazioni di coda hanno errori propri (richiesta duplicata, inesistente, non annullabile).
`src/app/api/prenotazioni/coda/route.ts`	Nessuna route esistente poteva ospitare elenco, ingresso e uscita dalla coda senza violare la responsabilità singola: è stata creata una route dedicata.
`src/hooks/use-ingresso-coda.ts`	La stessa logica di ingresso in coda serviva a mappa e griglia mobile (e il suo esito al messaggio di conflitto): è stata estratta in un hook condiviso.
`src/lib/eventi-coda.ts`	I nuovi eventi finiscono nella tabella `LogEvento` usata dalle metriche esistenti: il modulo elenca i tipi della coda da escludere dalle aggregazioni (a fine CR è usato solo dai test delle statistiche).
`src/app/notifiche/page.tsx`, `src/app/notifiche/tipo-config.tsx`	La pagina delle notifiche associava icone e colori solo ai tipi esistenti: i nuovi tipi richiedevano una configurazione con un ripiego per i tipi sconosciuti.
`src/middleware.ts`	La route della coda era già coperta dal matcher generale; è stata dichiarata anche in modo esplicito per evitare regressioni se il matcher venisse ristretto.
`src/components/admin/prenotazioni-actions.tsx`, `src/components/admin/statistiche-charts.tsx`	Lo staff deve sapere chi è stato promosso dopo un annullamento e vedere l'andamento della coda.
Migrazioni della lista d'attesa e di estensione degli enum	La tabella `ListaAttesa`, con l'indice FIFO e l'indice unico parziale sulle richieste `IN_ATTESA`, e i nuovi valori `CODA_*` degli enum `TipoNotifica` e `TipoEvento` sono stati introdotti in due passi distinti, ciascuno con la propria migrazione.

Tabella 3.6: Discovered Impact Set (DIS) per la CR02

Analisi: il DIS è ampio perché la CR02 ha seguito una strategia di estensione: componenti nuovi e dedicati, invece di modificare quelli esistenti. La verifica successiva ha mostrato un ulteriore punto sfuggito sia alla stima sia all'implementazione: nessuna schermata permetteva allo studente di vedere le proprie richieste in coda e di uscirne, benché l'API lo consentisse. È stata aggiunta in seguito una scheda dedicata in «Le mie prenotazioni».

3.5 Metrics

Valutiamo ora l'accuratezza della stima dell'impatto per questa Change Request utilizzando le metriche standard.

- **Candidate Impact Set (CIS):** 18 file
- **Actual Impact Set (AIS):** 22 file
- **Intersezione (CIS ∩ AIS):** 10 file

Precision (Precisione) Indica quanto la stima iniziale fosse mirata, ovvero quanta parte dei file previsti è stata effettivamente modificata.

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{10}{18} \approx 0{,}56\ (55{,}6\%)$$

Un valore del 56% indica una precisione appena superiore alla metà: la scelta di rappresentare la coda come entità separata ha reso superflue le modifiche a dashboard, scanner e calcolo dei giorni prenotabili, e il canale in tempo reale non è stato collegato; le altre due mancate modifiche (`src/app/api/prenotazioni/[id]/route.ts` e `src/app/api/admin/anomalie/route.ts`) sono omissioni dell'implementazione, corrette in seguito.

Recall (Richiamo) Indica quanto del lavoro finale effettivamente svolto era stato previsto all'inizio.

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{10}{22} \approx 0{,}45\ (45{,}5\%)$$

Un valore del 45% conferma che più di metà dell'implementazione ha riguardato file nuovi o componenti non previsti, come il servizio di dominio esteso, la route della coda e l'hook condiviso. Riflette la scelta, presa in corso d'opera, di aggiungere moduli dedicati invece di modificare quelli esistenti.